Lecture

10/20/26

- longest paths in directed graphs
  - NP complete problem, which means it is an intractable problem, requiring brute force algorithms and exponential time
- shortest path algorithm
  - Polynomial time algorithms
  - Shortest paths in Edit-like graphs can be solved optimally in linear time
- Edit-like graphs, DAG = directed acydic graph
- On DAGs, we can solve
  - Longest path } in linear time
  - Shortest path
  - Key concept: topological order

Dijkstra's Algorithm

$G = \langle N, A \rangle$ directed graph

N = the set of nodes/vertices

A = set of edges, each edge has a non-negative cost

$N = \{1, 2, \ldots, n\}$ where 1 is the source node

Problem: Given $G = \langle N, A \rangle$ and $L[i,j]$ = cost edge (i,j), compute the cost of the shortest paths from the source to each of the other nodes in the graph

C, S two sets of nodes

S = "special" nodes, nodes already chosen

C = the set of candidate nodes at any step S contains all the nodes whose minimal cost from the source node is already known

C = the rest of the nodes.

At any step we choose a candidate node in C where the cost to the source node is smallest cost and add it to S

Def A path from the source to a node v is special if all intermediate nodes are in S

Array D contains the cost of the shortest special path to each node in G

When we add a new node v from C to S, the shortest special path to v is also the shortest path in the graph

N = {1, 2, ..., n}
L[i,j] ≥ 0 if (i,j) ∈ A
L[i,j] = ∞ if (i,j) ∉ A

Initialization
S = {1}
C = {2, 3, ..., n}
For i = 2 to n do
    D[i] = L[1, i]

Greedy Loop
Repeat n-2 times
v = node in C with minimum value D[v]
S = S ∪ {v}
C = C \ {v}
For each w in C do
    if D[w] > D[v] + L[v, w]
        D[w] = D[v] + L[v, w]

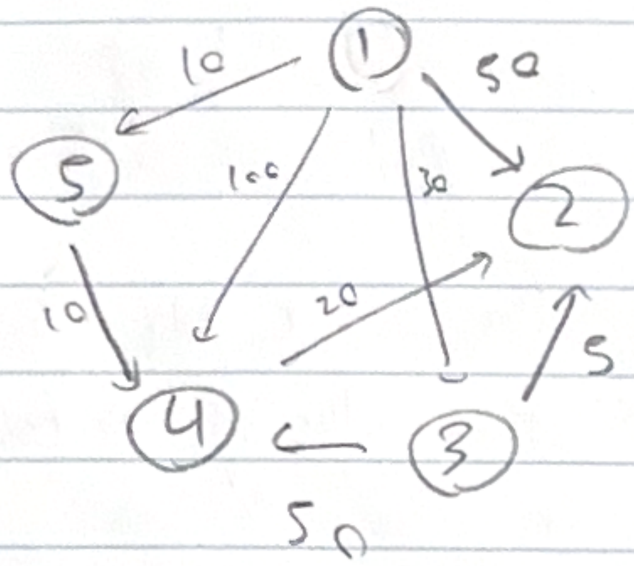
(3)

P[i] = 1

i = 2, 3, ..., n

P[2, 3 ... n]

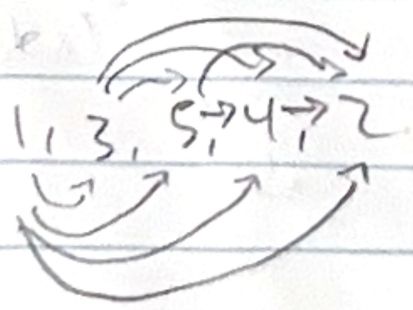
example:



step	v	CD	D
			2, 3, 4, 5
initialization	—	{2,3,4,5}	[50, 30, 100, 10]
1	5	{2,3,4}	[50, 30, 20, 10]
4	{2,3}		[40, 30, 20, 10]
3	{2}		[35, 30, 20, 10]

Topological sorting

A topological sort, or topological ordering, of a directed graph is a linear ordering of the vertices such that for every directed edge from vertex u to vertex v, vertex u comes before vertex v. A topological ordering is possible if and only if the graph is a DAG, i.e. it has no directed cycles. Topological ordering, if it exists, can be found in linear time



(4)

Topological sorting algorithm

For DAG G

L = the list of topological sorted vertices of V

S = Set of vertices with no incoming edges

Algorithm:

while $S \neq \emptyset$ do
  remove node u from S
  add u to L at the tail
  for each node v with edge $e = (u,v) \in E$
    remove edge e from graph
    if v has no incoming edges
      put v into S

ex/ edit graph

L: {1, 2, 4, 3, 5, 7, 6, 8, 9}

S: {~~1~~, ~~2~~, 4, 3

